

PROJET CALCUL PARALLELS :

Question :

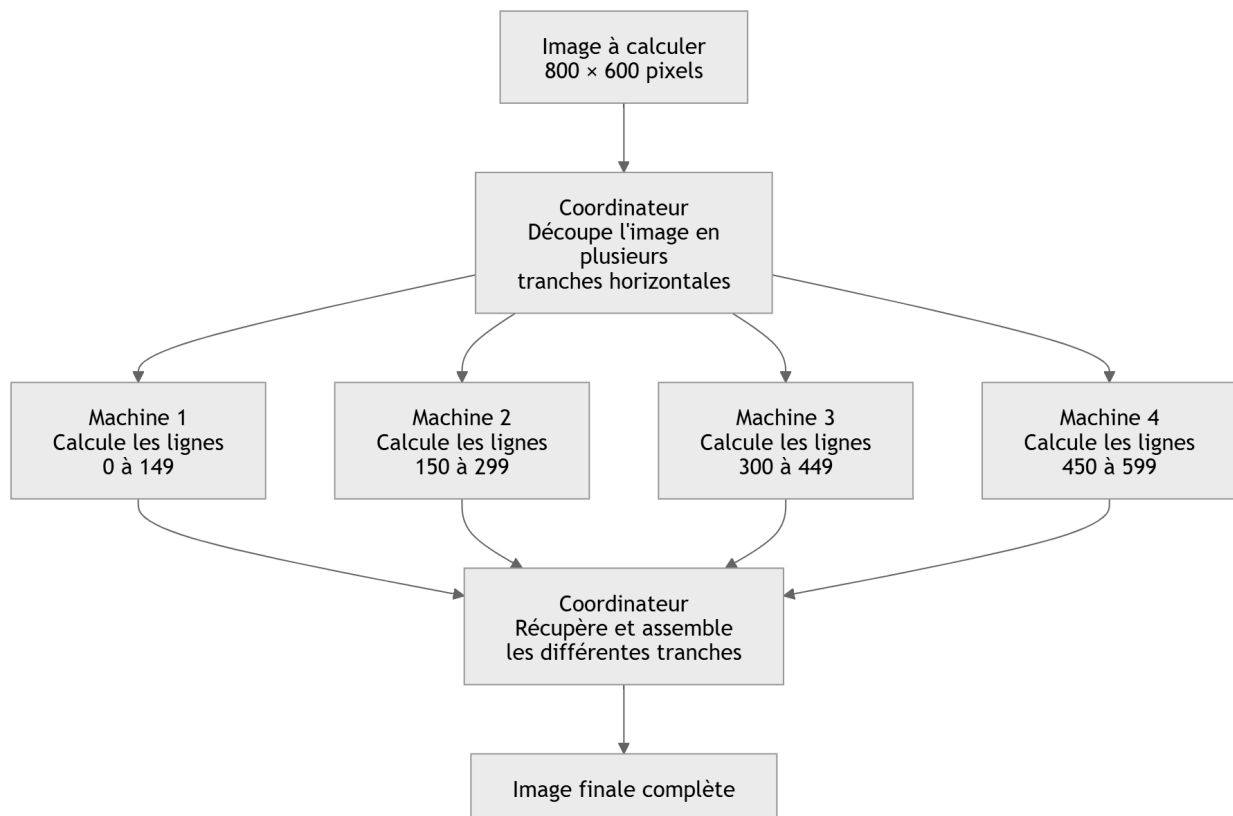
En utilisant votre meilleur outil, votre imagination, décrivez et illustrez comment cela pourrait être réalisé, sans rentrer dans les détails JAVA, que vous n'allez pas tarder à mettre en œuvre.

Une machine coordinatrice découpe l'image en plusieurs tranches horizontales, par exemple une tranche par machine disponible. Elle envoie ensuite à chaque machine de calcul la description de la scène ainsi que les coordonnées des lignes qu'elle doit traiter.

Chaque machine calcule indépendamment les pixels de sa tranche. Une fois son travail terminé, elle renvoie le résultat au programme chargé de l'assemblage.

Lorsque toutes les tranches ont été reçues, elles sont replacées dans le bon ordre afin de reconstituer l'image finale.

Schéma :



Questions :

1. Tester le programme en modifiant ses paramètres (sur la ligne de commande).

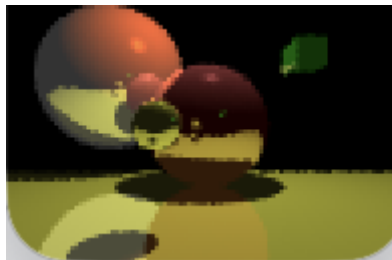
Le programme `LancerRaytracer.java` accepte trois paramètres en ligne de commande :

```
java LancerRaytracer [fichier-scène] [largeur] [hauteur]
```

- `fichier-scène` : le fichier décrivant la scène 3D (par défaut `simple.txt`)
- `largeur` et `hauteur` : les dimensions en pixels de l'image à calculer (par défaut 512×512)

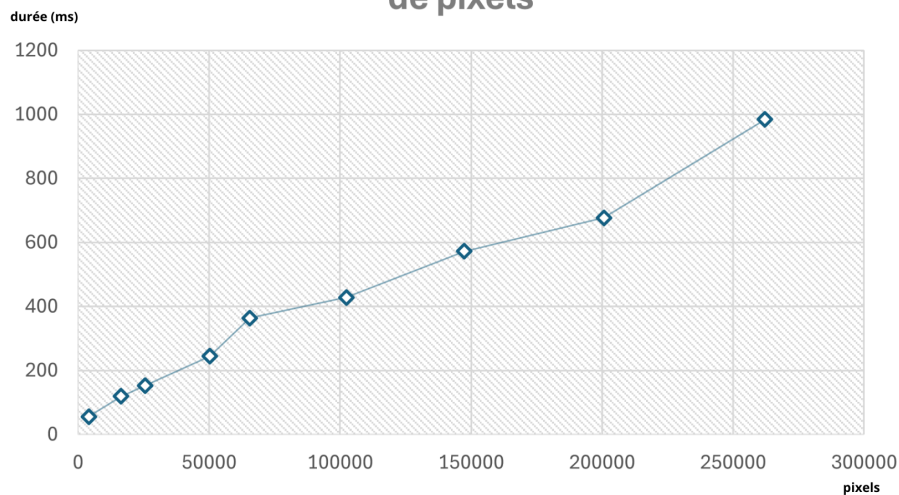
On peut donc modifier la résolution de l'image sans toucher au code, uniquement via la ligne de commande. Pour modifier la scène elle-même (objets, couleurs, lumières), on édite le fichier `simple.txt`.

Exemple : `java LancerRaytracer simple.txt 100 100`



2. Observer le temps de d'exécution en fonction de la taille de l'image calculée. Vous pouvez faire une courbe (temps de calcul et tailles d'image).

Durée de génération en fonction du nombre de pixels



Pour construire cette courbe, on a lancé plusieurs exécutions du raytracer avec des dimensions d'image différentes, de plus en plus grandes. À chaque exécution, le programme affiche dans la console le temps de génération de l'image, qu'on relève ensuite.

On fait ensuite un tableau avec, pour chaque test, le nombre de pixels de l'image et le temps de génération mesuré. Ce tableau sert ensuite à tracer la courbe temps de calcul / taille de l'image.

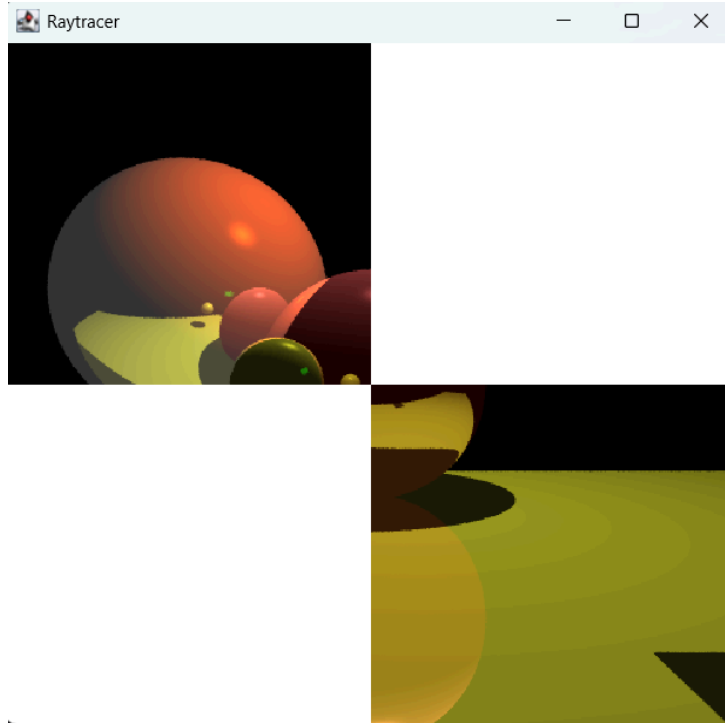
3. En ne modifiant **que** le fichier `LancerRaytracer.java`, reproduire l'image suivante :

Pour obtenir le rendu en échiquier, nous avons découpé notre image de 512x512 pixels en 4 carrés égaux (ou dalles) de 256x256 pixels.

Le point (0,0) se situe tout en haut à gauche. L'axe X avance vers la droite et l'axe Y descend vers le bas. Pour reproduire l'image, nous avons programmé le moteur de rendu pour qu'il calcule uniquement deux zones spécifiques :

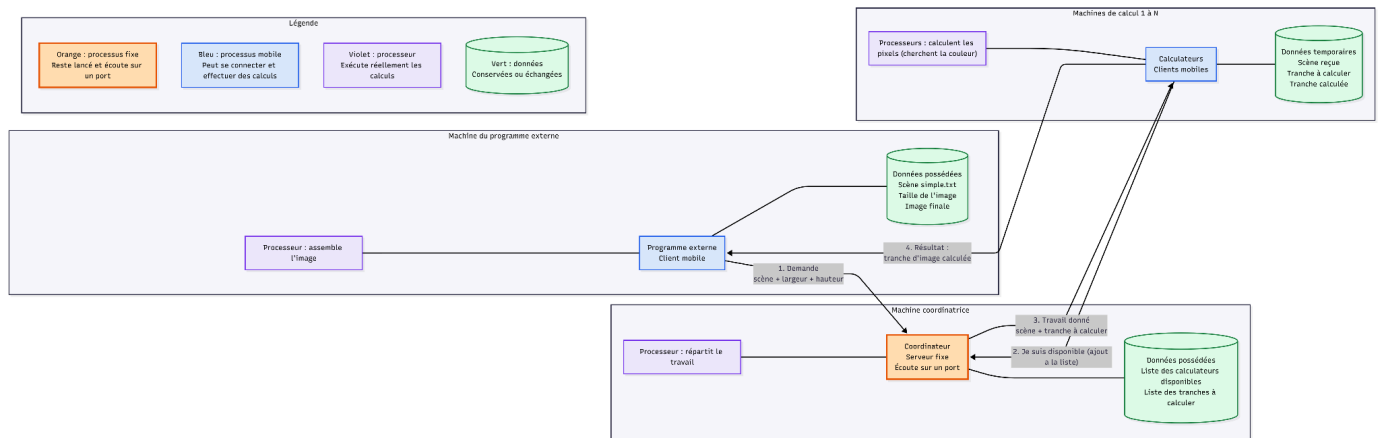
- Le carré haut-gauche : Il démarre à l'origine (0,0) et s'étend sur une largeur et une hauteur de 256 pixels.
- Le carré bas-droite : Son coin supérieur gauche commence au centre exact de l'image, soit aux coordonnées (256,256), et possède la même taille de 256x256 pixels.

Dans le code, nous avons supprimé la commande qui dessinait toute l'image d'un coup. À la place, nous appelons la fonction de calcul deux fois de suite avec ces coordonnées ciblées. Les deux autres carrés de l'image ne sont jamais calculés, ce qui les laisse blancs par défaut et crée le motif demandé.



Faire un petit schéma de cette architecture en identifiant les choses suivantes :

Schéma :



Si on veut que les calculs se fassent en parallèle, que faut-il faire ?

Pour que les calculs se fassent en parallèle, il faut déjà découper les données : l'image globale (ici 512x512) doit être divisée en plusieurs sous-images indépendantes (par exemple, si on reprend ce que l'on a fait précédemment, 4 dalles de 256x256). Un serveur fixe centralisé, le coordinateur, répertorie toutes les machines de calcul disponibles et stocke la liste des tranches à calculer.

Enfin, les appels RMI étant bloquants par défaut (c'est-à-dire qu'un programme s'arrête en faisant un appel synchrone et attend passivement que la machine distante ait terminé son traitement avant de passer à la ligne suivante), le coordinateur doit lancer chaque demande de calcul vers les nœuds dans un thread séparé. Pour orchestrer cela efficacement et s'adapter au nombre de machines réelles, le coordinateur utilise un pool de threads. En créant ces fils d'exécution indépendants, le coordinateur distribue les tâches simultanément (via des appels asynchrones) à tous les calculateurs disponibles, sans bloquer son propre programme. Chaque nœud de calcul distant reçoit ainsi sa tranche de travail et l'exécute localement sur son propre CPU.

Au fur et à mesure que les calculateurs renvoient leurs résultats, le coordinateur assemble de manière synchronisée les tranches d'images reçues pour reconstruire l'image finale, qu'il renverra ensuite au client. Par rapport à la version séquentielle où une seule machine calcule tout l'un après l'autre, le temps total est ici divisé par le nombre de processeurs distants actifs, moins le coût de transport des données sur le réseau. Le calcul est donc radicalement accéléré.

Résumé du projet et interface :

1. Pseudo-code des interfaces

L'architecture repose sur deux contrats (interfaces) qui permettent aux machines de communiquer à distance (via RMI) :

Interface ServiceCalculateur (Distant) :

- Fonction calculerTranche(scene, x, y, largeur, hauteur) retourne Image : Permet au coordinateur de commander le calcul d'un morceau d'image.
- Fonction getNom() retourne Chaîne : Permet d'identifier l'ouvrier (utile pour les logs dans le terminal).

Interface ServiceCoordinateur (Distant) :

- Fonction enregistrerCalculateur(nom, calculateur) retourne Entier : Permet aux ouvriers de s'annoncer au démarrage.

- Fonction `retirerCalculateur(nom)` : Permet aux ouvriers de se déconnecter.
- Fonction `getNombreCalculateurs()` retourne Entier
- Fonction `lancerLeRendu(scene, largeur, hauteur)` retourne Image : Permet au client de lancer la génération globale.

2. Fonctionnement de notre architecture RMI

Notre implémentation s'articule autour de trois rôles :

Le Coordinateur (Serveur RMI Fixe) :

Il maintient un registre actif des ouvriers disponibles.

Lorsqu'il reçoit une requête `lancerLeRendu` d'un client, il détermine le nombre N d'ouvriers enregistrés et découpe la hauteur de l'image totale en N tranches égales.

Pour contourner le blocage inhérent à RMI, il instancie N `Thread` (fils d'exécution). Chaque thread se connecte à un ouvrier distant en même temps et appelle `calculerTranche(...)`.

Le coordinateur met ensuite son programme principal en pause (avec un `join()`) en attendant que tous ses Threads aient terminé leur mission, puis il fusionne les sous-images reçues (via une méthode `copierPixels`) dans l'image finale qu'il retourne au client.

Les Calculateurs (Ouvriers mobiles) :

Ils se connectent au Coordinateur au démarrage et invoquent `enregistrerCalculateur` pour lui donner leur référence.

Ils attendent ensuite qu'on les sollicite. Lorsque `calculerTranche` est invoqué par le coordinateur, ils utilisent le moteur de rendu de raytracer (`scene.compute`) sur leur propre processeur local et renvoient le résultat.

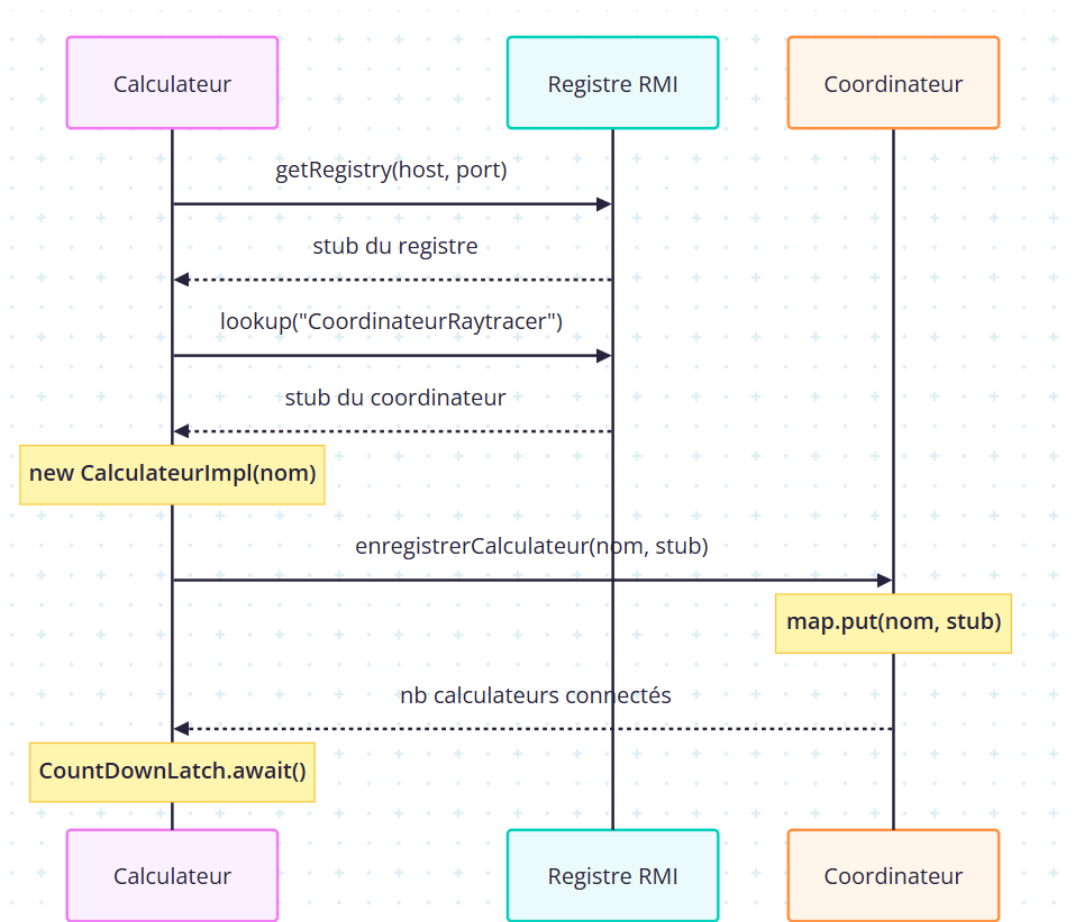
Le Client (Demandeur) :

Il charge le fichier de description de la scène (`simple.txt`), se connecte au Coordinateur via le registre RMI local, et invoque `lancerLeRendu(...)`. Il mesure le temps total écoulé, sauvegarde et affiche la fenêtre finale de l'image générée en parallèle.

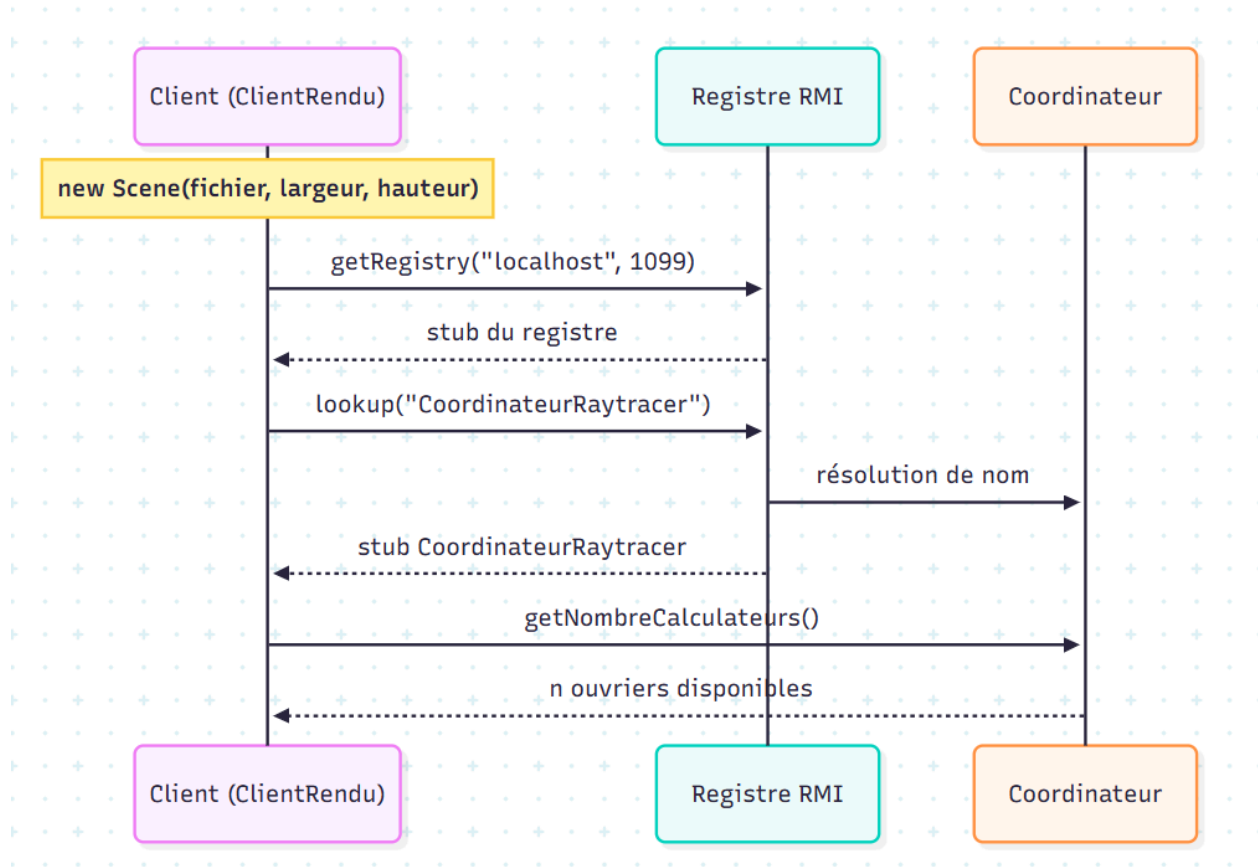
Duree par nombre de calculateurs		
nb_calculateurs		duree_generation (ms)
	1	7332
	2	5283
	3	4118
	5	3895

Avec les résultats obtenus, on observe que plus le nombre d'ouvriers est élevé, plus la génération de l'image est rapide.

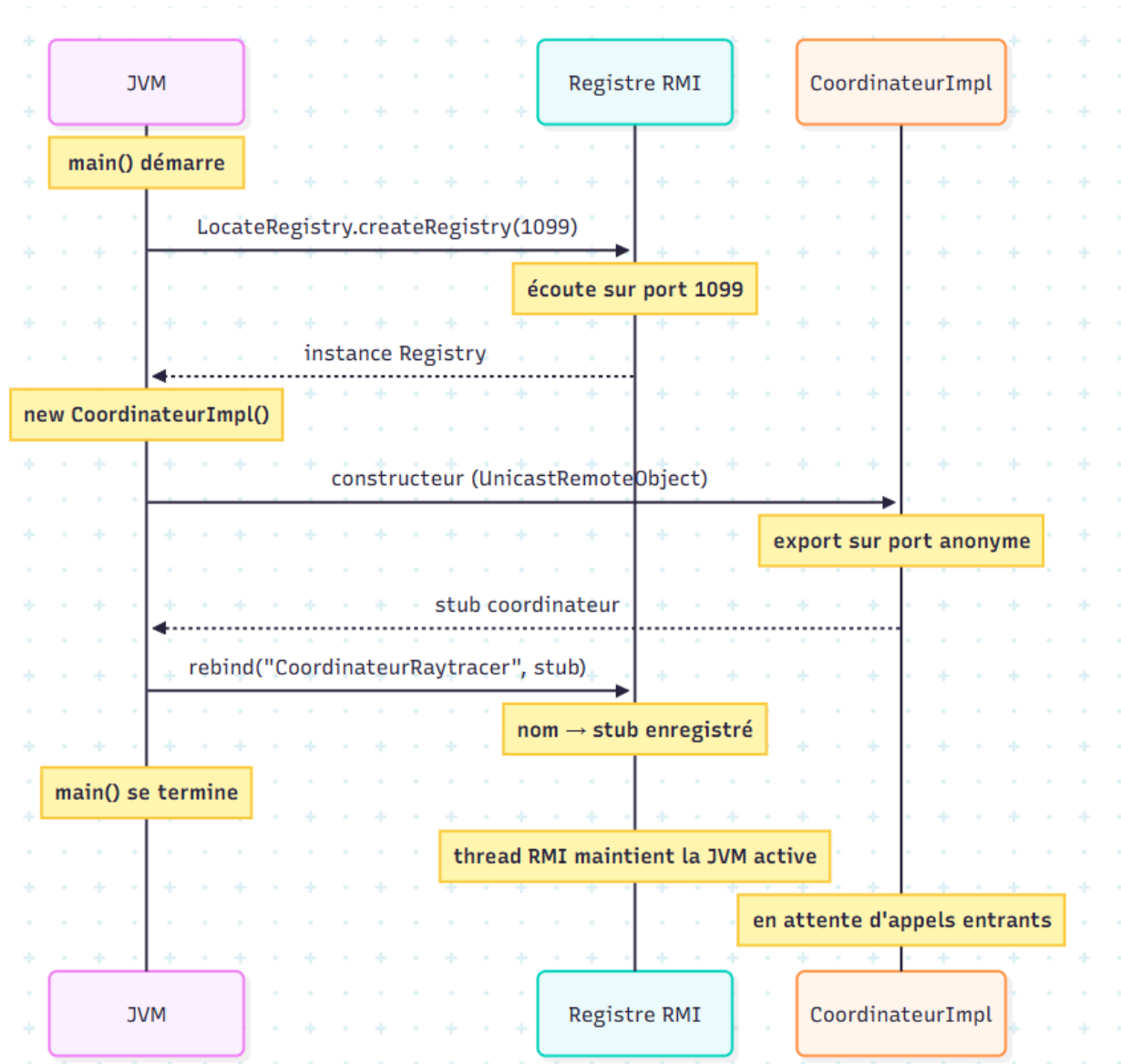
Un calculateur se connecte au coordinateur



Un client se connecte au coordinateur



LanceurCoordinateur



Rendu Parallèle Complet

